Personal Development

Semester 2

Derjen Frick
NHL STENDEN 5588472

Contents

Goals

During Semester 2, we had to draw up a Personal Development Plan (PDP) tailored more towards our development for our upcoming Internship period. I focused more on my coding strength and knowledge, as I want to expand my abilities and knowledge when it comes to programming.

Deploy Production Ready Website

The goal is to go beyond just building projects locally and actually ship something to production. This means setting up a proper CI/CD pipeline on GitHub, containerizing the app with Docker, and having it live and accessible. The focus is on production-level code quality — proper error handling, core CRUD functionality, and a clean basic UI. At least 4 hours a week should go into this. The end result should be a deployed application that demonstrates real-world development skills, not just a local demo.

Build and Integrate an AI Chatbot

Building on the app from Goal 1, the plan is to integrate an AI chatbot that can answer basic questions and help with simple problem solving. This involves creating backend API endpoints to handle chatbot communication and building a frontend UI for it. About 2 hours a week goes into research and learning AI integration. The measurable result is a working chatbot visible on the website that can handle basic user queries.

Learn Swift

The goal is to step outside of web development and get hands-on with mobile by learning Swift. Following tutorials for at least 2 hours a week, the plan is to build a functional Swift app with at least 3 screens, covering navigation, buttons, and state management. The end result is a working iOS app that can be demoed or shown through screenshots as proof of completion.

Results

The good news is that I have achieved all 3 Goals, and in this chapter I will showcase these 3 goals.

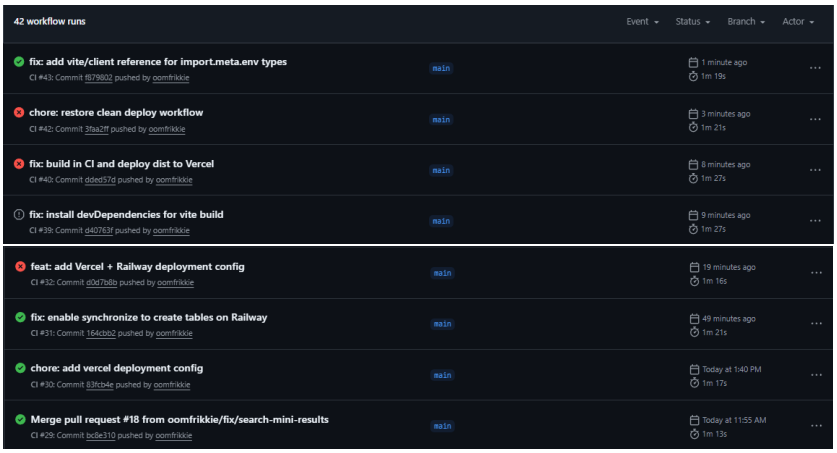
Deploy production ready website

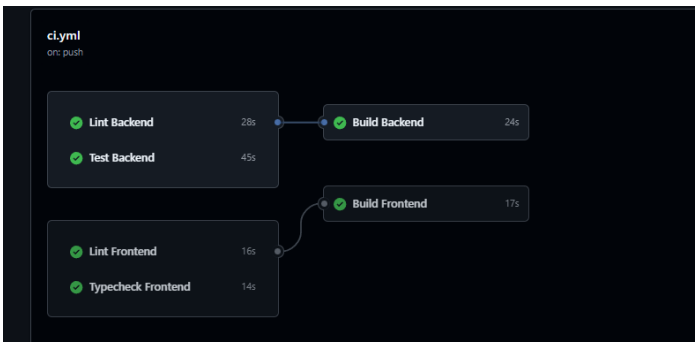
In this chapter I will use screenshots showcasing everything from the CICD to the AI.

CI / CD

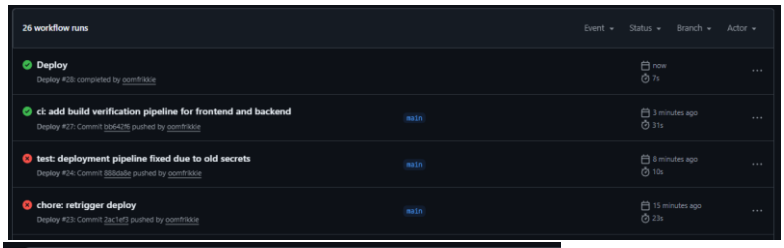
This chapter will showcase the CI/CD pipeline in my github repository.

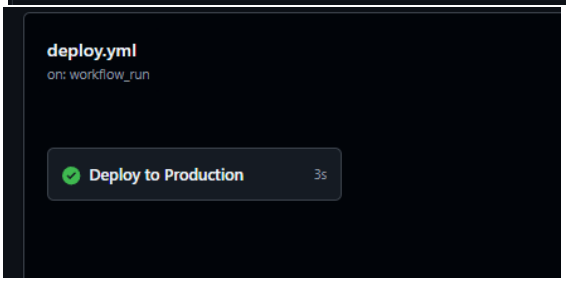
CI:





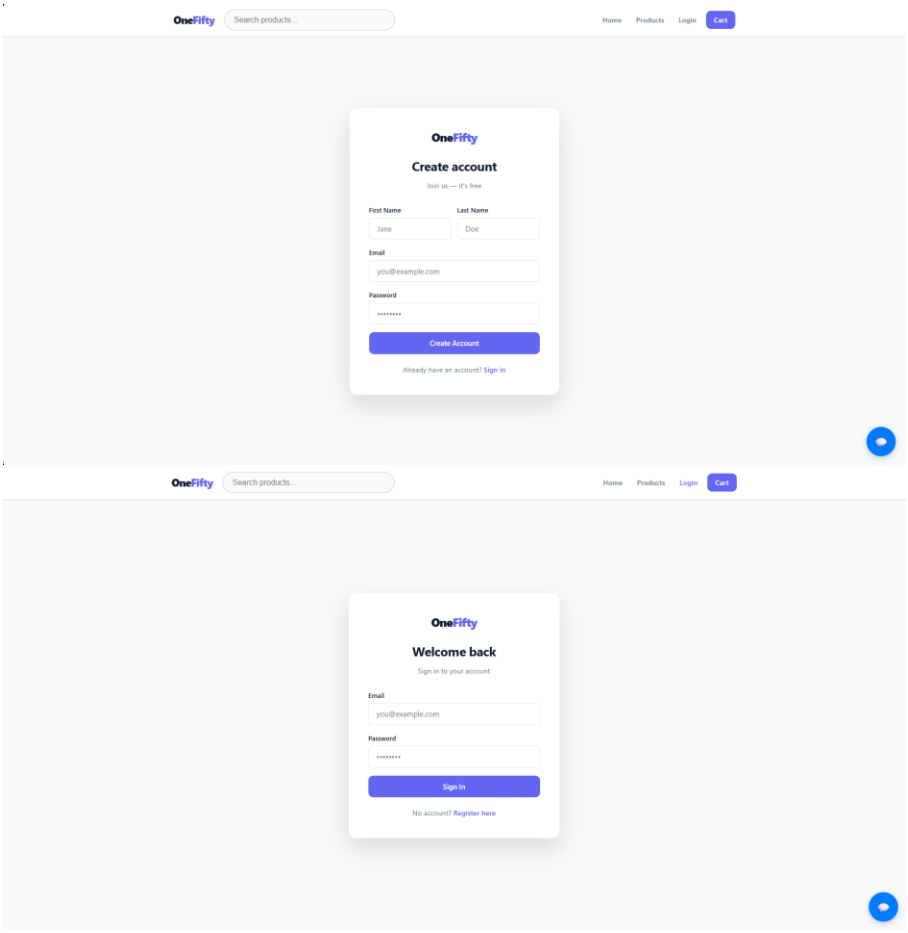
CD:

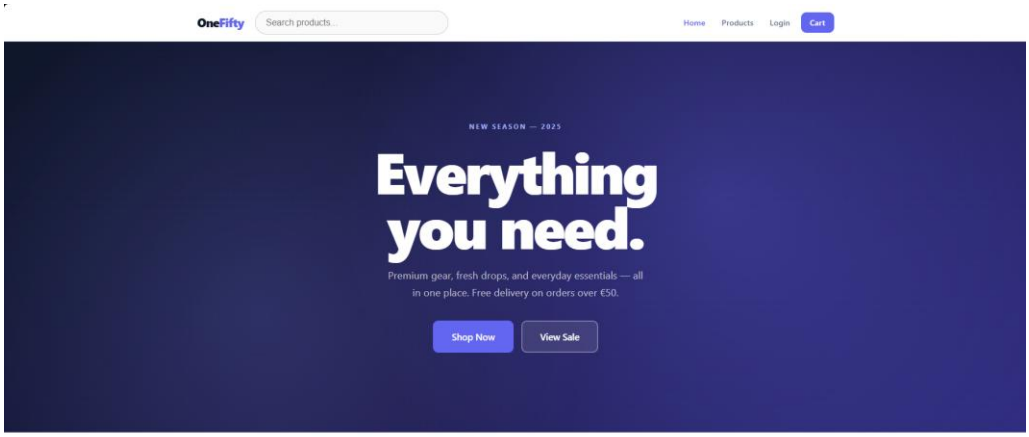

26 workflow runs
Event
Status
Branch
Actor
Deploy
Deploy #28: completed by oomfrikkie
now
7s
ci: add build verification pipeline for frontend and backend
Deploy #27: Commit bb642f6 pushed by oomfrikkie
main
3 minutes ago
31s
test: deployment pipeline fixed due to old secrets
Deploy #24: Commit 888da8e pushed by oomfrikkie
main
8 minutes ago
10s
chore: retrigger deploy
Deploy #23: Commit 2ac1ef3 pushed by oomfrikkie
main
15 minutes ago
23s

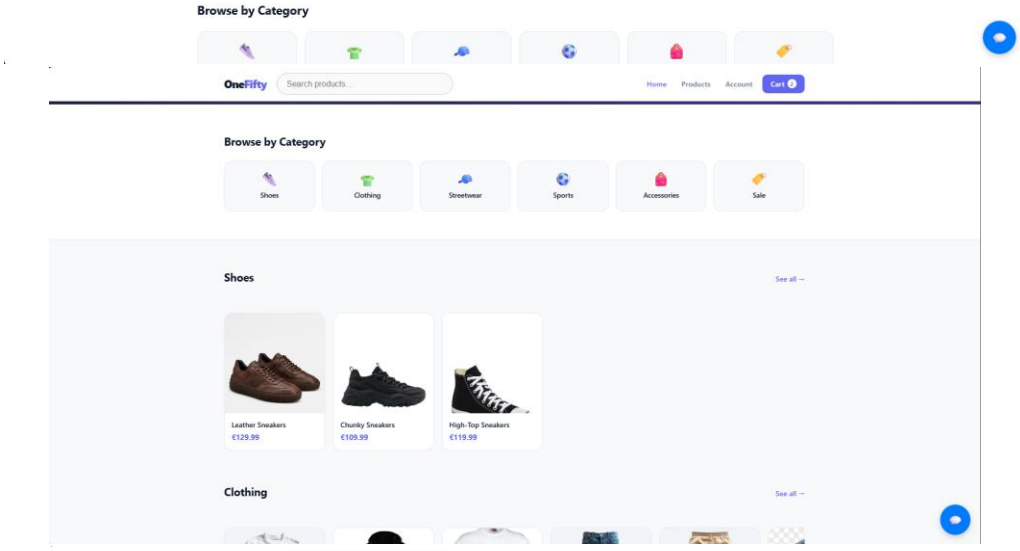

deploy.yml
on: workflow_run
Deploy to Production
3s

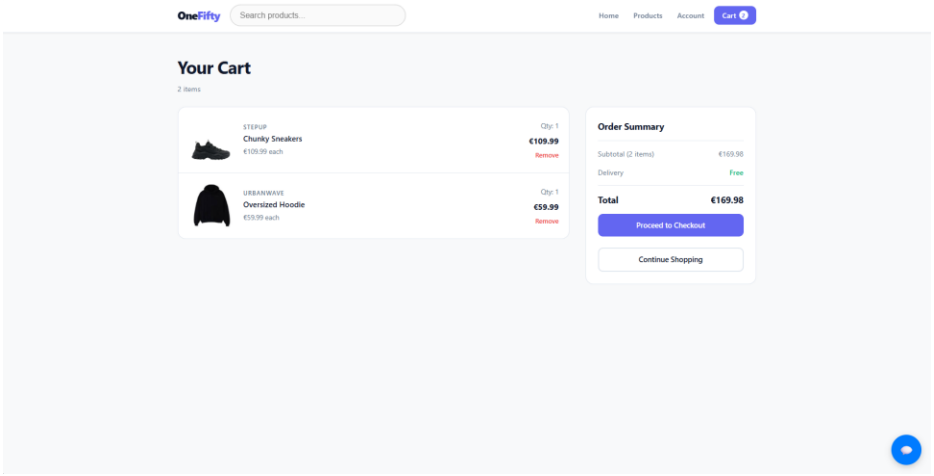
Website

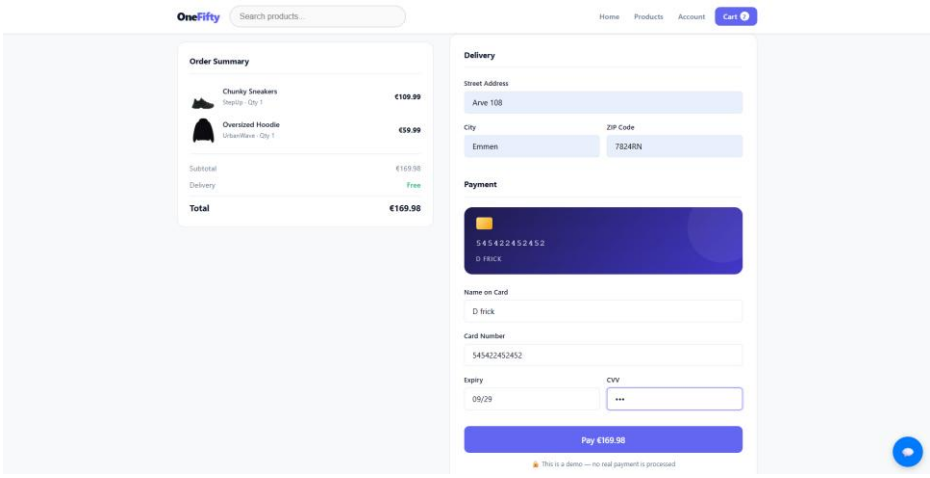
In this chapter I will showcase the website itself

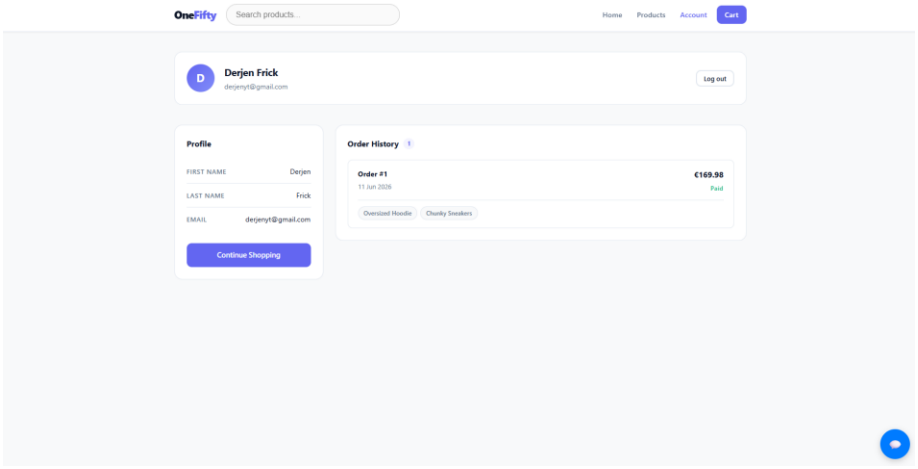



OneFifty
Search products...
Home
Products
Login
Cart
NEW SEASON — 2025
Everything you need.
Premium gear, fresh drops, and everyday essentials — all in one place. Free delivery on orders over €50.
Shop Now
View Sale


Browse by Category
OneFifty
Search products...
Home
Products
Account
Cart
Browse by Category
Shoes
Clothing
Streetwear
Sports
Accessories
Sale
Shoes
See all →
Leather Sneakers
€129.99
Chunky Sneakers
€109.99
High-Top Sneakers
€119.99
Clothing
See all →


OneFifty
Search products...
Home
Products
Account
Cart
Your Cart
2 items
STEPUP
Chunky Sneakers
€109.99 each
Qty: 1
€109.99
Remove
URBANWAVE
Oversized Hoodie
€59.99 each
Qty: 1
€59.99
Remove
Order Summary
Subtotal (2 items)
€169.98
Delivery
Free
Total
€169.98
Proceed to Checkout
Continue Shopping





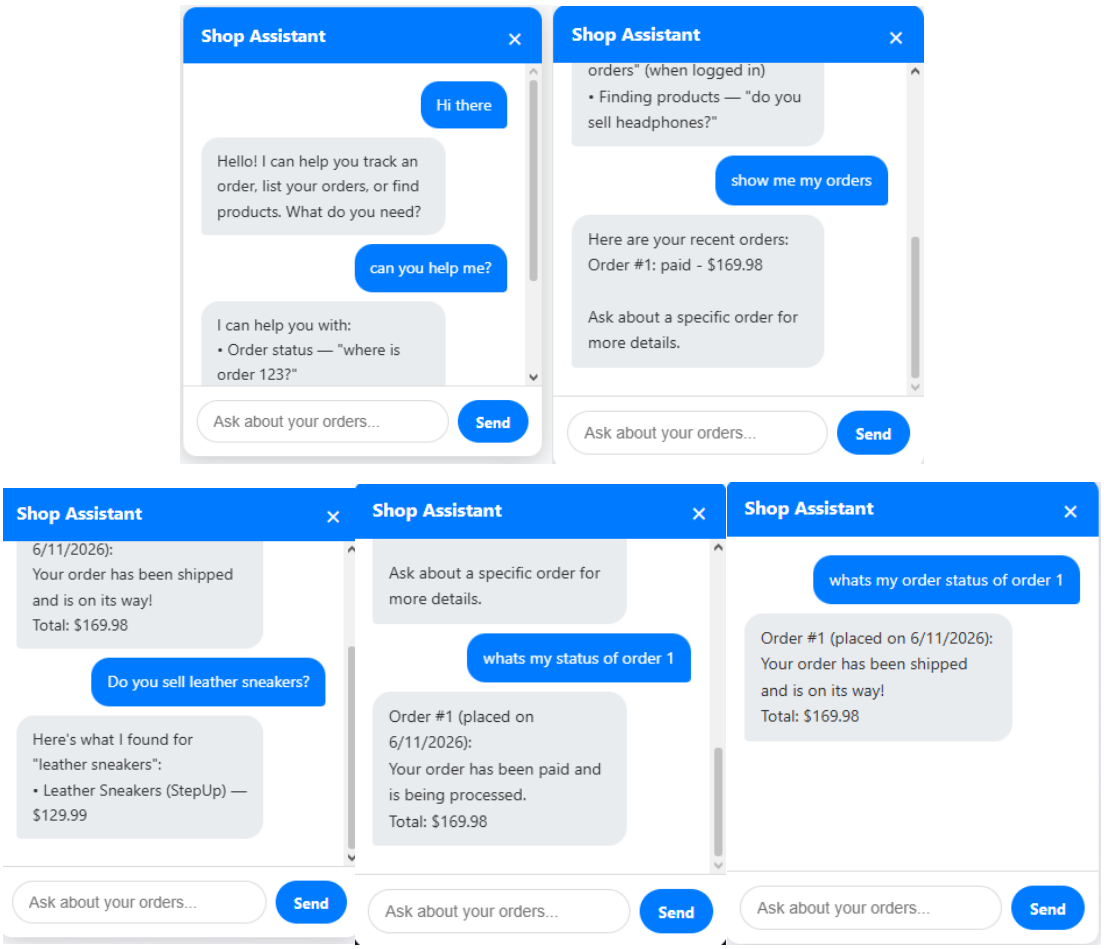
Website link:

https://ecommerce-website-vert-phi-10.vercel.app/home

Explanation on what I learnt:

So I built a full-stack ecommerce site from scratch, React and TypeScript on the frontend with Vite, NestJS on the backend, PostgreSQL with TypeORM for the database. Covered the whole thing, auth with email verification, cart, orders, admin, the works. Containerized everything with Docker Compose so the frontend, backend and database all run together locally. Then set up a GitHub Actions pipeline that lints, type checks, runs tests against a real Postgres instance and builds both apps on every push, and once that passes it automatically deploys the frontend to Vercel and the backend to Railway. Basically went through the full software development lifecycle end to end.

Implement AI Chat bot



Explanation on what I learnt:

So basically I built a chatbot for an e-commerce backend, but not the fake kind that just does if message.includes("order"). An actual machine learning model, Naive Bayes, written from scratch in TypeScript with no libraries.

The whole thing works in 4 steps: clean the text, classify the intent, pull out the details like an order number, then hit the database and reply. The "AI" part is honestly just counting. It reads a bunch of

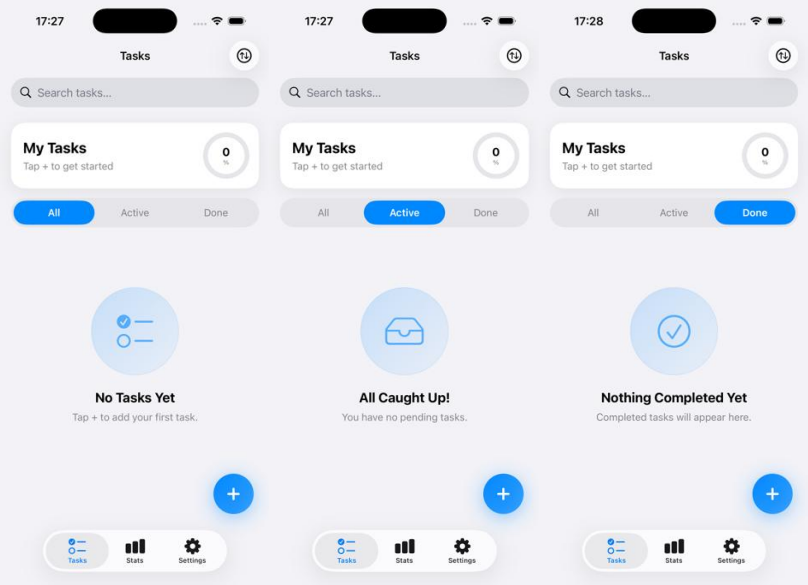
example sentences, tallies up which words show up in which intent, and when a real message comes in it scores it against all the intents and picks the best match.

Wrote it all in NestJS so it fits right into how I normally build backends. The classifier trains once on startup and when it's not confident enough (below 40%) it just asks for clarification instead of guessing wrong, which is actually a nice pattern to know.

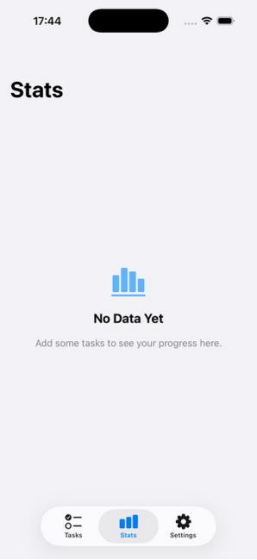
Create Swift App

I set out to further my knowledge by creating a basic task app in iOS language named swift, in this chapter I will showcase it with photos and all the features.

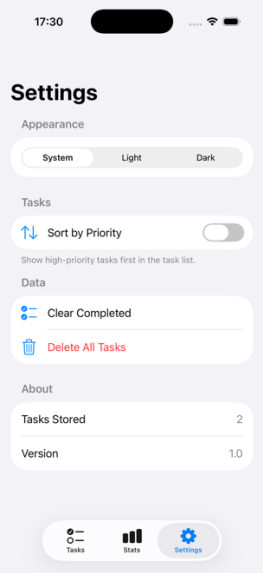
Tasks Screen:



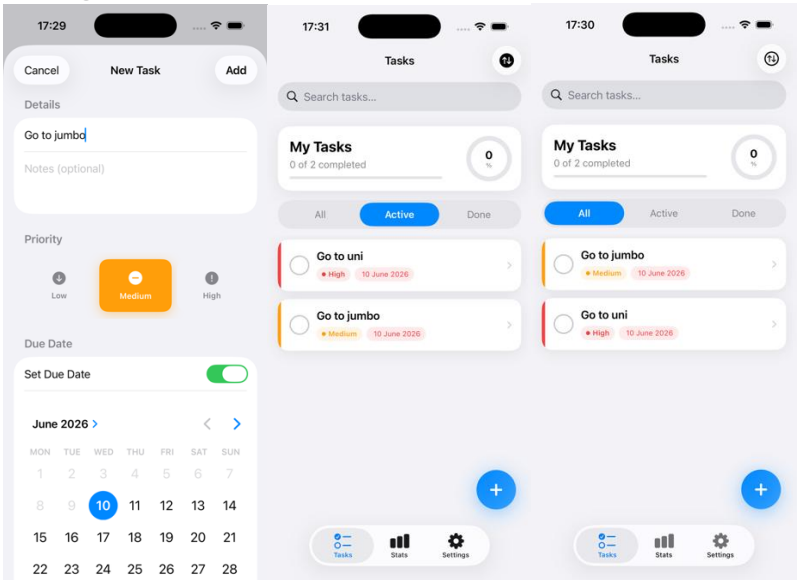
Stats screen(Empty)



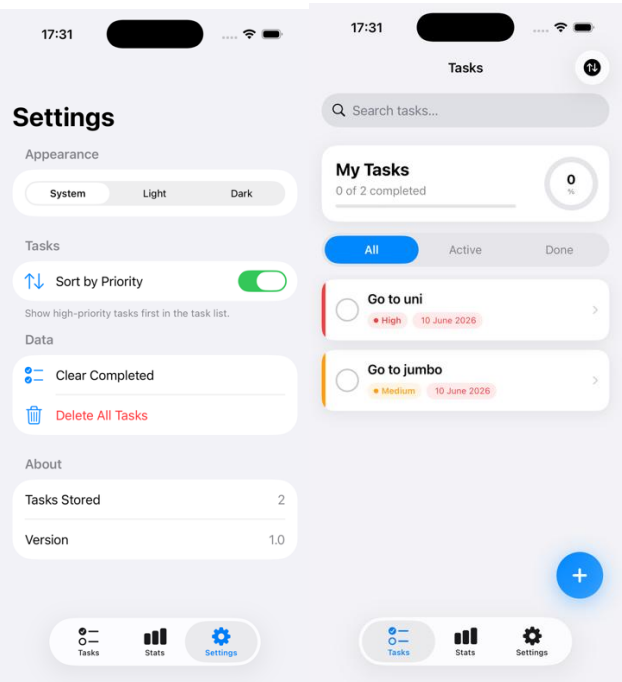
Settings Screen:



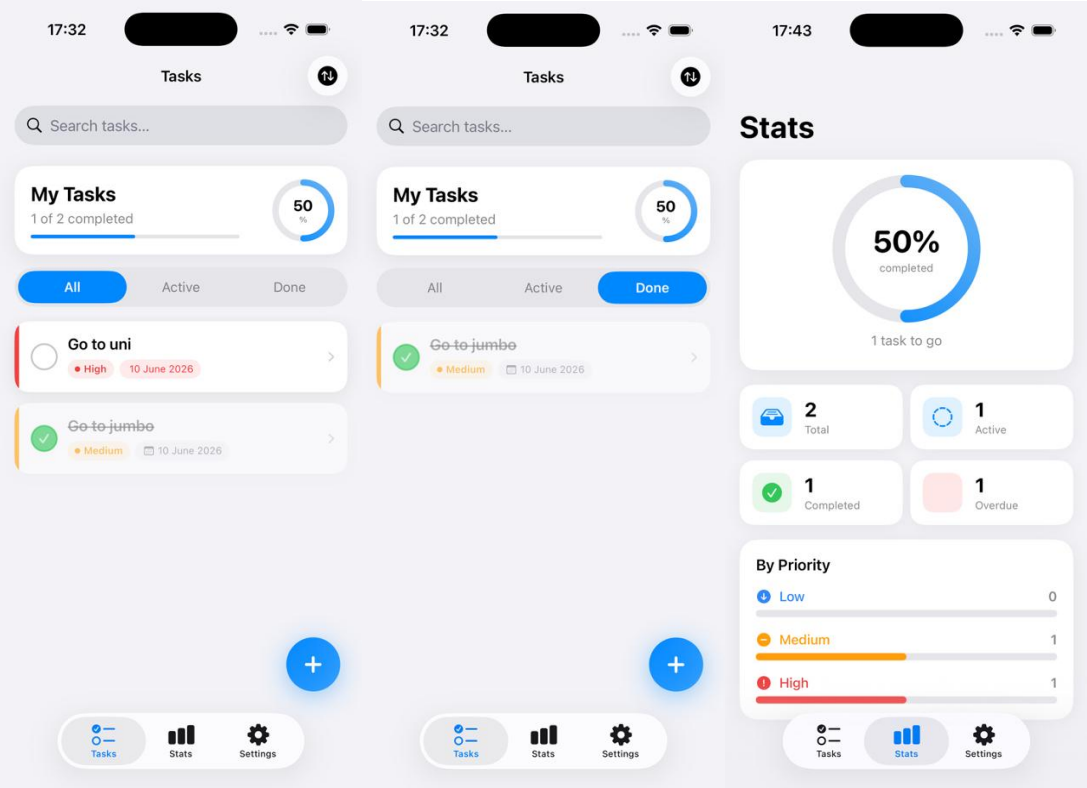
Adding a task:



Arranging by priority:



Completing a task



Explanation on what I learnt

So I built a task manager app in Swift and learned a lot from it. I got comfortable with how to structure a UI properly, breaking things into reusable components instead of one giant messy file, and doing layouts with stacks and grids. The big thing was figuring out state management, basically how to manage data across the whole app so everything stays in sync no matter which tab you're on. Also set up real data persistence so tasks actually save between sessions, added proper navigation with sheets and tabs, and polished it up with some animations and a progress ring. Main takeaway was keeping things cleanly separated, your data, your logic, and your UI, and not overcomplicating how you manage state.

Conclusion

Overall I would say I fully achieved my goals possible and I am happy with my progress. Going into this I wanted to build real projects that covered the full stack and I did exactly that. From a backend with a proper database and authentication system, to a React frontend, to setting up Docker and automated deployments, to even picking up Swift and building a mobile app. I also built a working AI chatbot from scratch, no libraries, just pure machine learning logic written in TypeScript, which was probably the most interesting thing I built. Every project taught me something new and I could see myself improving with each one. I did not just follow tutorials, I built things from scratch, ran into real problems and figured them out. That is what I am most proud of.